How can several people change one system without losing intent or trust?

Student notes for lecture 7. Follow a change from saved files to shared history, read a commit graph, reconcile conflicting intentions, inspect a clean merge with contradictory behavior, build a useful review packet, compare integration points, reproduce a result from its inputs, and preserve the reason for a decision. A separate history shows how bisection locates a regression.

Scenes in this lecture

#	SCENE (STUDENT HEADING)	ROLE	DEEP LINK
01	A branch name moves. A recorded commit stays put.	Anchor	`week-07.html#/commit-graph`
02	Removing the conflict can remove the rule.	Anchor	`week-07.html#/conflict-and-intent`
03	Different files can still disagree about one promise.	Bridge	`week-07.html#/clean-merge-wrong-behavior`
04	A diff needs a claim you can check.	Anchor	`week-07.html#/review-with-context`
05	When will the incompatible assumption become visible?	Bridge	`week-07.html#/integration-frequency`
06	The source is the same. Why did the answer change?	Bridge	`week-07.html#/same-code-different-result`
07	Will “fix stuff” help you understand this change later?	Bridge	`week-07.html#/history-as-explanation`
08	Which missing evidence blocks this change?	Bridge	`week-07.html#/ready-to-integrate`

Campus Rooms and the dependency teaching-range are fictional teaching examples. The 90-minute policy is temporary; the canonical 30-to-120-minute inclusive range returns at the end. Code is Python-like pseudocode.

The merge is green. Which limit should you believe?

Ari changes the booking rule. Bo adds a message. They edit different files in Campus Rooms.

Ari edits policy.py; Bo edits messages.py. Minus removes a line; plus adds a line.

POLICY · TEMPORARY R-02 VARIANT

```
- MAX_MINUTES = 120
+ MAX_MINUTES = 90
```

MESSAGE

```
def limit_message():
    return "Choose 30 to 120 minutes"
```

Prediction: if the edits combine, is the behavior ready? The text merges cleanly. A 100-minute request returns “Rejected by R-02”, while the message says “Choose 30 to 120 minutes.” The message and the policy describe different behavior.

> A shared change needs an intended behavior, a history, focused review, and checks of the combined result. The 90-minute rule is temporary.

The software has contracts. The change needs context.

Week 6 followed requests across boundaries. Now those promises must survive several people changing the system.

What you will be able to explain

- Read a small commit graph and distinguish commits, branch references, and merges.
- Explain the difference between a textual conflict and an incompatible behavioral change.
- Identify the context and evidence needed to review a change.
- Explain how dependency and configuration choices affect reproducibility.
- Use history and decision records to explain why a change was made.

Recall

Week 6 established request and result contracts, safe retry identity, and decisions with revisit triggers.

Case baseline

R-02 accepts 30 to 120 minutes inclusive. R-03 keeps cancellation with the booking owner. A temporary shorter-booking trial; restore the approved 120-minute baseline afterward.

Saved on your machine is different from shared with the team

Follow one policy edit through four places. Each action changes a different part of the record.

1. Working tree

Save the edit. Files saved on your machine; further edits can still change them.

2. Selected changes

Select this change. The next snapshot is prepared here. In Git this is the staging area.

3. Local commit

Record locally. A commit records the prepared snapshot and its parent. Your branch reference moves.

4. Shared history

Share the commit. Pushing sends commits to a remote and requests its branch update. Your remote-tracking reference records the observed position.

The edit and the references

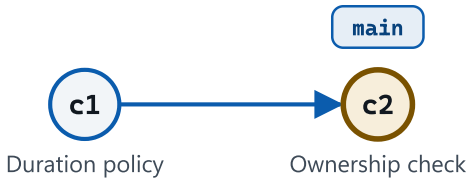
policy.py: MAX_MINUTES = 120 becomes MAX_MINUTES = 90. The local policy-90 reference moves from c2 to c3 when the commit is recorded. origin/policy-90 stays at c2 until sharing updates it. An unsaved editor buffer is outside this strip.

A branch name moves. A recorded commit stays put.

Panel 1 of 4 · The common ancestor and its parent

Condition: **Two lines of work from the same ancestor** State: **3 of 7**

Which part can move: a recorded snapshot, its parent relationship, or a branch name? The branch name moves to the next recorded commit.



C2 · RECORDED CONTENT

`MAX_MINUTES = 120`

Add cancellation ownership check

No duration message in this snapshot.

HOW TO READ THIS PICTURE

Circle: a recorded commit

Arrow: parent to child

Pill: a movable branch name

Left to right: teaching order; read the parent links

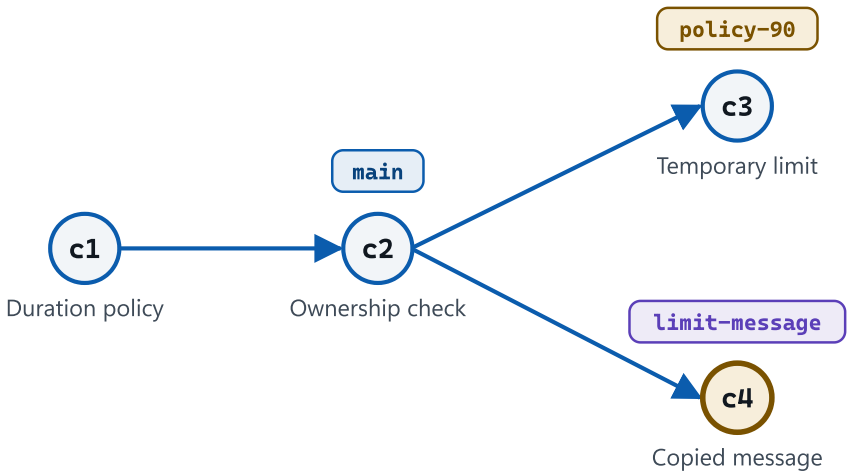
> A commit records a snapshot and its parent relationship. A branch name points to a commit; a later commit can move that name.

A branch name moves. A recorded commit stays put.

Panel 2 of 4 · Two descendants of the same ancestor

Condition: **Two lines of work from the same ancestor** State: **5 of 7**

c4: limit-message develops from **c2**. The descendants share an ancestor; neither contains the other.



C4 · RECORDED CONTENT

```
MAX_MINUTES = 120
```

Show duration limit message (hard-coded 120)

Message limit: 120 minutes.

HOW TO READ THIS PICTURE

Circle: a recorded commit

Arrow: parent to child

Pill: a movable branch name

Left to right: teaching order; read the parent links

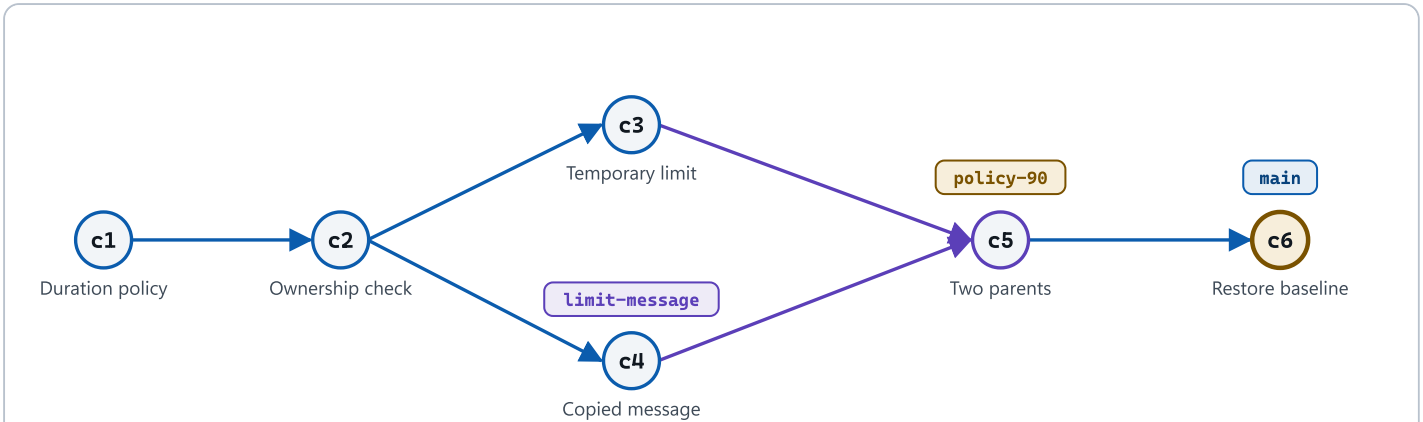
c3 and c4 both descend from c2. One changes the policy; the other adds a copied message. Neither descendant includes the other.

A branch name moves. A recorded commit stays put.

Panel 3 of 4 · A two-parent merge and the later restoration

Condition: **Two lines of work from the same ancestor** State: **7 of 7**

c5 connects c3 and c4 as a two-parent merge. The later c6 records the restoration and main moves to it. All six recorded commits remain in the history.



C6 · RECORDED CONTENT

`MAX_MINUTES = 120`

Restore canonical 120 limit

Message limit: 120 minutes.

HOW TO READ THIS PICTURE

Circle: a recorded commit

Arrow: parent to child

Pill: a movable branch name

Left to right: teaching order; read the parent links

The restoration is another recorded change. Moving a name preserves the earlier commits and their parent links.

At c5, the 90-minute policy and 120-minute message disagree. c6 restores the canonical 120-minute limit; the readiness scene checks that restoration. A merge connects ancestry; its combined behavior needs its own checks.

Principle · History records relationships between changes; branch names are movable references, not the changes themselves.

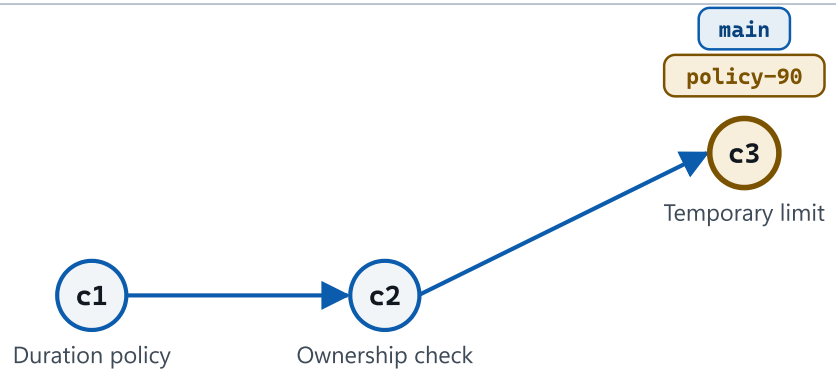
In short · Follow the parents; watch the names move.

A branch name moves. A recorded commit stays put.

Panel 4 of 4 · Changed: no divergence, so a reference can fast-forward

Condition: **The current tip is an ancestor of the other tip** State: **3 of 4**

main moves from **c2 to c3**. This is a **fast-forward**; it adds no merge commit.



C3 · RECORDED CONTENT

`MAX_MINUTES = 90`

Variant: max duration 90 (labeled temporary)

No duration message in this snapshot.

HOW TO READ THIS PICTURE

Circle: a recorded commit

Arrow: parent to child

Pill: a movable branch name

Left to right: teaching order; read the parent links

main at c2 moves to c3 without a merge commit when nothing diverged.

main at c2 is an ancestor of c3. Moving main to c3 needs no new merge commit. The shorter limit remains a labeled temporary variant.

Removing the conflict can remove the rule.

Panel 1 of 3 · Original and edits: two intentions touch one validation

Condition: **The resolution loses the upper bound** State: **2 of 5**

Which intentions must survive? Keep the temporary shorter policy and the clearer validation structure. Read the shared original and both edits before examining the resolution.

SHARED ANCESTOR

```
1 MAX_MINUTES = 120
2 def duration_ok(minutes):
3     if minutes < 30:
4         return False
5     if minutes > MAX_MINUTES:
6         return False
7     return True
```

EDIT A · TEMPORARY POLICY

```
1 MAX_MINUTES = 90
2 def duration_ok(minutes):
3     if minutes < 30:
4         return False
5     if not minutes <= MAX_MINUTES:
6         return False
7     return True
```

EDIT B · RESTRUCTURE

```
1 MAX_MINUTES = 120
2 def duration_ok(minutes):
3     return (
4         minutes >= 30
5         and minutes <= MAX_MINUTES
6     )
```

R-02 · TEMPORARY VARIANT

A booking lasts 30 to 90 minutes, both included.

Edit A implements this limit. Edit B keeps the same rule in a clearer shape.

> The policy changes from 120 to a temporary 90-minute maximum; the other edit rewrites the inclusive range check. A textual conflict asks how to combine these intentions.

Removing the conflict can remove the rule.

Panel 2 of 3 · A valid-looking resolution loses the upper bound

Condition: **The resolution loses the upper bound** State: **4 of 5**

91, 120 and 121 minutes are accepted. R-02 rejects each of them: the temporary upper bound has disappeared.

SHARED ANCESTOR

```
MAX_MINUTES = 120
def duration_ok(minutes):
    if minutes < 30:
        return False
    if minutes > MAX_MINUTES:
        return False
    return True
```

EDIT A · TEMPORARY POLICY

```
MAX_MINUTES = 90
def duration_ok(minutes):
    if minutes < 30:
        return False
    if not minutes <= MAX_MINUTES:
        return False
    return True
```

EDIT B · RESTRUCTURE

```
MAX_MINUTES = 120
def duration_ok(minutes):
    return (
        minutes >= 30
        and minutes <= MAX_MINUTES
    )
```

R-02 · TEMPORARY VARIANT

A booking lasts 30 to 90 minutes, both included.

Edit A implements this limit. Edit B keeps the same rule in a clearer shape.

CHOSEN RESOLUTION

```
MAX_MINUTES = 90
def duration_ok(minutes):
    return 30 <= minutes
```

Python-like pseudocode: the return value answers whether the duration is allowed.

BOUNDARY COMPARISON · TEMPORARY VARIANT

MINUTES	RESOLUTION DOES	R-02 REQUIRES	AGREE?
29	Rejected	Rejected	✓ Yes
30	Accepted	Accepted	✓ Yes
90	Accepted	Accepted	✓ Yes
91	Accepted	Rejected	✗ No
120	Accepted	Rejected	✗ No
121	Accepted	Rejected	✗ No

The shown resolution accepts 91, 120 and 121 minutes. Each violates the temporary R-02 maximum. Syntax and marker removal answer a different question from policy behavior.

Removing the conflict can remove the rule.

Panel 3 of 3 · Keep the new rule and the clearer structure

Condition: **The resolution preserves both intentions** State: **4 of 4**

The resolution preserves the new rule and the structural improvement.

SHARED ANCESTOR

```
MAX_MINUTES = 120
def duration_ok(minutes):
    if minutes < 30:
        return False
    if minutes > MAX_MINUTES:
        return False
    return True
```

EDIT A · TEMPORARY POLICY

```
MAX_MINUTES = 90
def duration_ok(minutes):
    if minutes < 30:
        return False
    if not minutes <= MAX_MINUTES:
        return False
    return True
```

EDIT B · RESTRUCTURE

```
MAX_MINUTES = 120
def duration_ok(minutes):
    return (
        minutes >= 30
        and minutes <= MAX_MINUTES
    )
```

R-02 · TEMPORARY VARIANT

A booking lasts 30 to 90 minutes, both included.

Edit A implements this limit. Edit B keeps the same rule in a clearer shape.

CHOSEN RESOLUTION

```
MAX_MINUTES = 90
def duration_ok(minutes):
    return 30 <= minutes <= MAX_MINUTES
```

Python-like pseudocode: the return value answers whether the duration is allowed.

BOUNDARY COMPARISON · TEMPORARY VARIANT

MINUTES	RESOLUTION DOES	R-02 REQUIRES	AGREE?
29	Rejected	Rejected	✓ Yes
30	Accepted	Accepted	✓ Yes
90	Accepted	Accepted	✓ Yes
91	Rejected	Rejected	✓ Yes
120	Rejected	Rejected	✓ Yes
121	Rejected	Rejected	✓ Yes

The resolution retains MAX_MINUTES = 90 and both inclusive comparisons. The table covers the lower bound, the changed upper bound, and durations beyond it.

Principle · A merge resolution must reconcile intent and verify the combined behavior, not merely remove conflict markers.

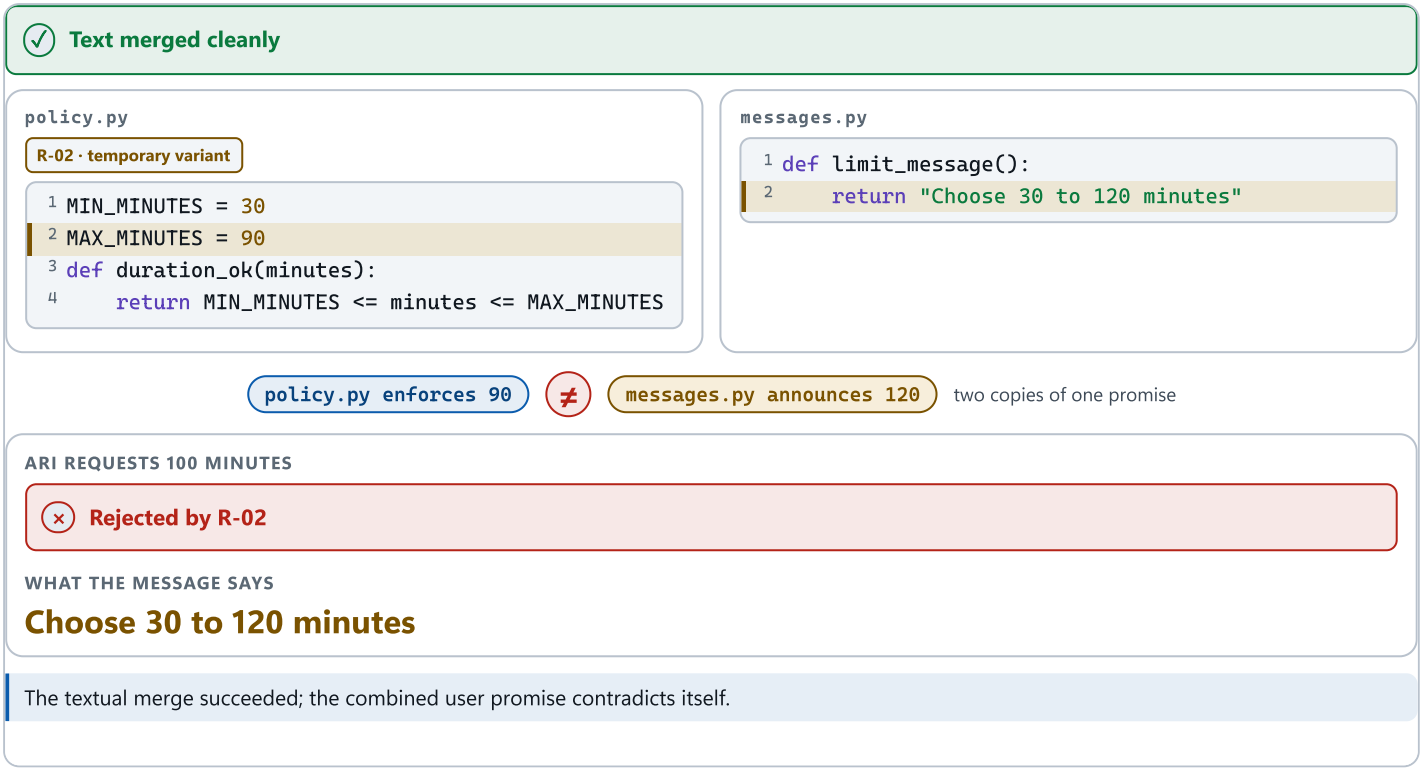
In short · Keep both intentions, then check the boundary.

Different files can still disagree about one promise.

Panel 1 of 2 · A clean merge, followed by contradictory behavior

Condition: **The message stores a second limit** State: **4 of 4**

Two copies of the same decision have drifted. Inspect the combined user experience.



The two files combine without a textual conflict. A 100-minute booking is rejected under the temporary 90-minute rule, while the message still offers up to 120 minutes.

Principle · Textual integration and semantic integration are different kinds of evidence.

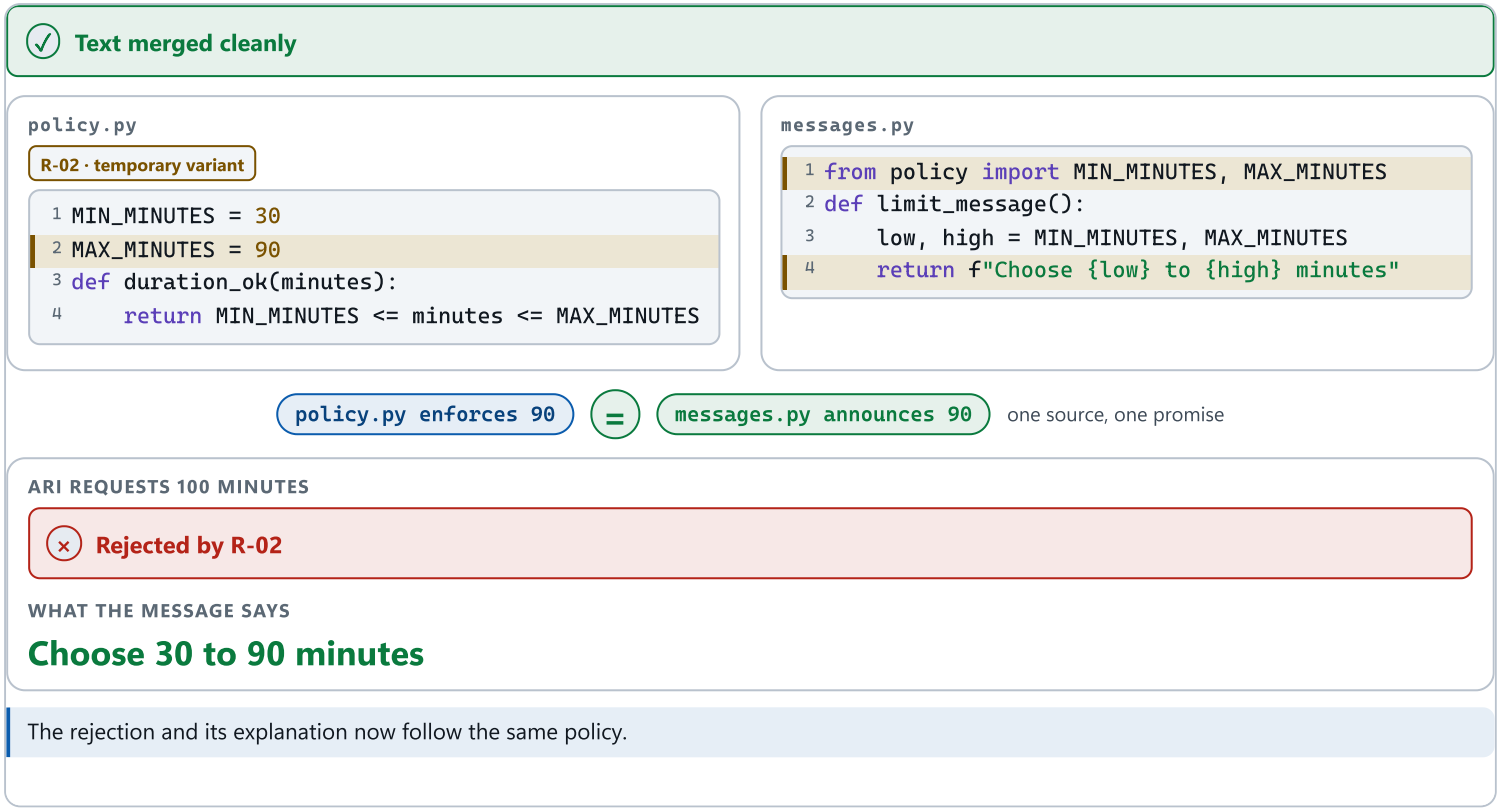
In short · Check the promise across the combined files.

Different files can still disagree about one promise.

Panel 2 of 2 · Changed: the message reads the same approved policy

Condition: **The message reads the approved policy** State: **4 of 4**

One policy source connects the user message to the behavior it explains.



Keep the same request and the same temporary policy. Deriving the message from Policy produces “Choose 30 to 90 minutes,” which agrees with the rejection.

Principle · Textual integration and semantic integration are different kinds of evidence.

In short · Check the promise across the combined files.

A diff needs a claim you can check.

Panel 1 of 4 · The mixed diff with its purpose and unresolved concerns

Condition: **The policy change includes an unrelated rename** State: **3 of 6**

What can you decide from a diff alone? You can see the edits. To judge them, connect the intended behavior, constraints and checks to the actual change.

THE PROPOSED CHANGE · − REMOVES, + ADDS

Change by Ari; review by Bo.

```
- MAX_MINUTES = 120
+ MAX_MINUTES = 90
```

`Message: “Choose 30 to 120 minutes”.`

Also included: an unrelated helper rename.

WHAT THE REVIEWER NEEDS

Purpose	R-02 variant: 90-minute upper limit (temporary, labeled)
Constraint	A temporary shorter-booking trial; restore the approved 120-minute baseline afterward.
Alternatives	Copy the limit into every message / Read the approved limit from Policy

CHECKS ON THIS COMBINED CHANGE

✓ Boundary table: pass: 90 accepted, 91 rejected.
× Message agrees with Policy: fail: says 120.
? R-03: combined-change cancellation results are missing.

Linters and formatters handle routine style. Human review examines correctness, security, design, and meaning.

REVIEW THE CHANGE, WITH A REASON

Correctness	Required	The message still says 120; derive it from the policy value.
Security	Required	Retain R-03. Show the owner and non-owner cancellation results.
Design	Required	The helper rename is unrelated; split it into its own change.
Understandability	Resolved	State why the variant exists and when it is restored.
Optional preference	Optional	MAX_DURATION_MINUTES would read better.

A SUBSTANTIVE CONCERN STAYS OPEN

The duration change preserves cancellation ownership. Missing evidence: Owner and non-owner cancellation results on the combined change.

> The boundary table passes, but the message disagrees with Policy and cancellation evidence is missing. Required comments address the change and name the reason. An optional naming preference is labeled optional.

A diff needs a claim you can check.

Panel 2 of 4 · Changed: review the unrelated rename separately

Condition: **The unrelated rename is reviewed separately** State: **5 of 6**

Ari can cancel B-001; Bo is rejected. The combined change retains ownership. The substantive disagreement now has evidence and a decision owner.

THE PROPOSED CHANGE · – REMOVES, + ADDS

Change by Ari; review by Bo.

```
- MAX_MINUTES = 120
+ MAX_MINUTES = 90
```

`Message: derive the displayed limit from Policy.`

The helper rename is in its own change.

WHAT THE REVIEWER NEEDS

Purpose	R-02 variant: 90-minute upper limit (temporary, labeled)
Constraint	A temporary shorter-booking trial; restore the approved 120-minute baseline afterward.
Alternatives	Copy the limit into every message / Read the approved limit from Policy

CHECKS ON THIS COMBINED CHANGE

✓ Boundary table: pass: 90 accepted, 91 rejected.
✓ Message agrees with Policy: says 90.
✓ R-03: Ari can cancel B-001; Bo is rejected; the ownership check is retained.

Linters and formatters handle routine style. Human review examines correctness, security, design, and meaning.

REVIEW COMMENTS · ORIGINAL CONCERNS

Correctness Resolved	The message still says 120; derive it from the policy value.
Security Resolved	Retain R-03. Show the owner and non-owner cancellation results.
Design Separate change	The helper rename is unrelated; split it into its own change.
Understandability Resolved	State why the variant exists and when it is restored.
Optional preference Optional	MAX_DURATION_MINUTES would read better.

EVIDENCE RESOLVES THE CONCERN

Both results are linked to the change; the approval explains the retained R-03 check. Ari owns the change decision; Bo reviews the evidence.

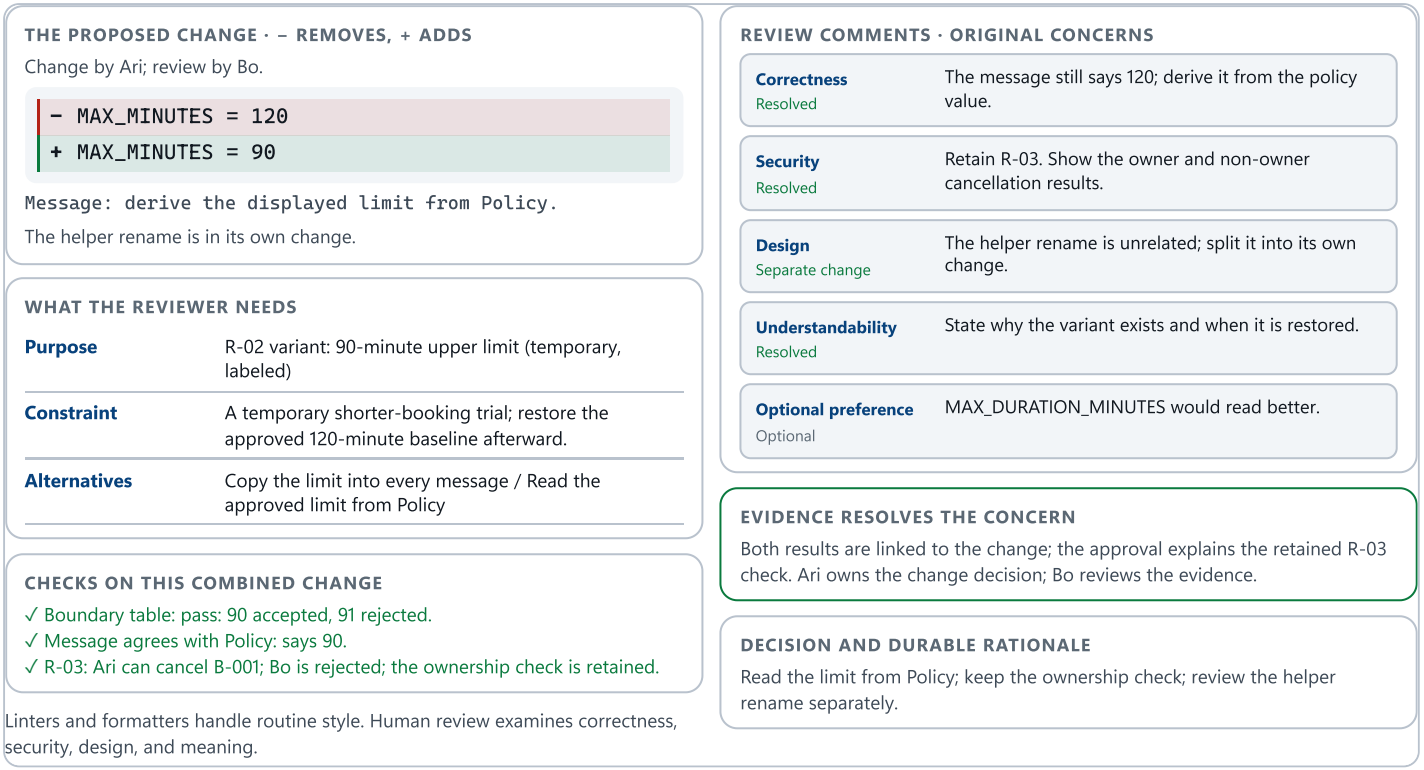
> A focused change keeps the policy claim and its checks together. The message now follows Policy. Required authorization evidence shows that Ari can cancel B-001 and Bo is rejected; the ownership check remains.

A diff needs a claim you can check.

Panel 3 of 4 · The review decision survives the conversation

Condition: **The unrelated rename is reviewed separately** State: **6 of 6**

Asynchronous review or a short paired investigation can resolve missing context. Keep the claim, checks, decision owner, and rationale with the change.



Restate the claim, identify missing evidence, record who decides, and preserve the rationale. A short paired investigation can establish missing context; asynchronous comments can do the same when their findings remain with the change.

Principle · Review is an evidence-based conversation about a change, not a vote on how convincing its author sounds.

In short · Make the purpose and the checks visible.

A diff needs a claim you can check.

Panel 4 of 4 · Changed: the same diff is labeled as an assistant draft

Condition: **The same mixed diff was drafted by an assistant** State: **3 of 6**

Classify the comments by their reason. Each one addresses the change, not the person. **Required correctness and security concerns** differ from an **optional preference**.

THE PROPOSED CHANGE · – REMOVES, + ADDS

Drafted by an automated assistant. Description: “fix stuff”.

```
- MAX_MINUTES = 120
+ MAX_MINUTES = 90
```

`Message: “Choose 30 to 120 minutes”.`

Also included: an unrelated helper rename.

WHAT THE REVIEWER NEEDS

Purpose	R-02 variant: 90-minute upper limit (temporary, labeled)
Constraint	A temporary shorter-booking trial; restore the approved 120-minute baseline afterward.
Alternatives	Copy the limit into every message / Read the approved limit from Policy

CHECKS ON THIS COMBINED CHANGE

✓ Boundary table: pass: 90 accepted, 91 rejected.
× Message agrees with Policy: fail: says 120.
? R-03: combined-change cancellation results are missing.

Linters and formatters handle routine style. Human review examines correctness, security, design, and meaning.

REVIEW THE CHANGE, WITH A REASON

Correctness	Required	The message still says 120; derive it from the policy value.
Security	Required	Retain R-03. Show the owner and non-owner cancellation results.
Design	Required	The helper rename is unrelated; split it into its own change.
Understandability	Resolved	State why the variant exists and when it is restored.
Optional preference	Optional	MAX_DURATION_MINUTES would read better.

A SUBSTANTIVE CONCERN STAYS OPEN

The duration change preserves cancellation ownership. Missing evidence: Owner and non-owner cancellation results on the combined change.

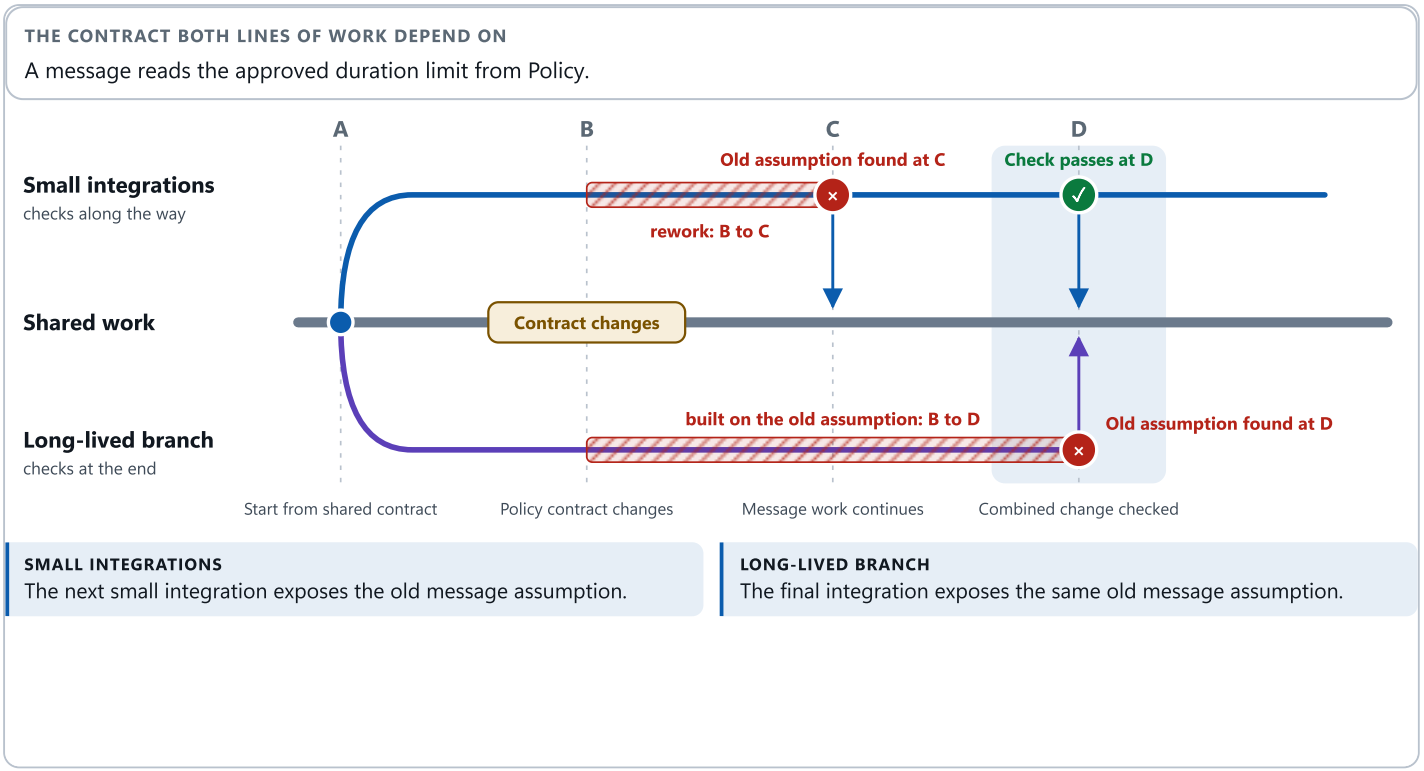
> The description “fix stuff” omits the intended behavior and constraints. The reviewer still needs the same requirement, checks, and rationale. Authorship changes the label, not the review questions.

When will the incompatible assumption become visible?

Panel 1 of 3 · The contract changes before the message work is finished

Condition: **The shared message contract changes during the work** State: **4 of 5**

At **D**, the long-lived branch finally meets the same changed assumption. More work was built before that feedback arrived.



Both timelines begin with the same contract and make the same intended changes. The changed contract appears at B. Small integrations expose the old assumption at C; the long-lived branch finds it at D, after more work was built on it. The letters mark the order of events, not elapsed time.

Principle · Choose a collaboration strategy that exposes incompatible assumptions early enough to act on them.

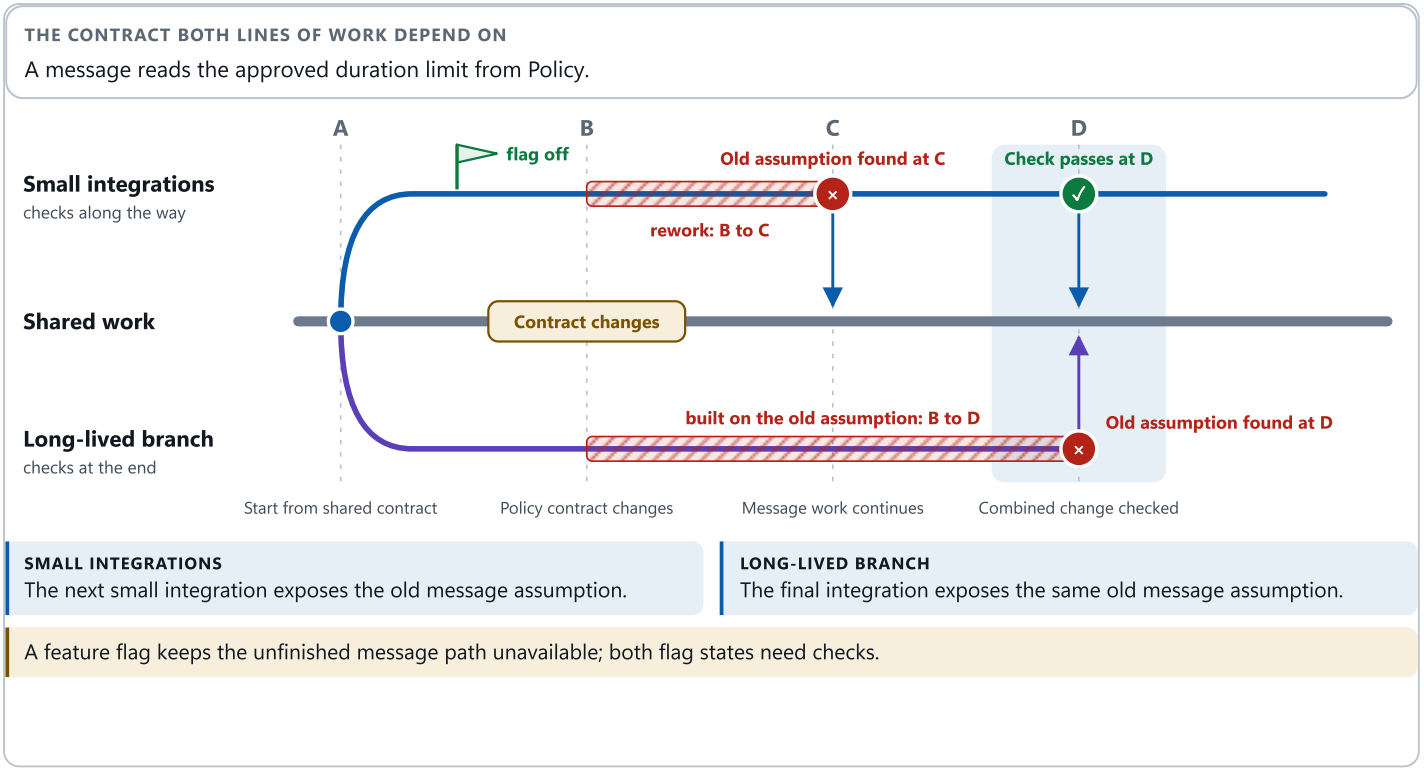
In short · Meet changed assumptions while the work is small.

When will the incompatible assumption become visible?

Panel 2 of 3 · Smaller integrations still need a safe unfinished path

Condition: **The shared message contract changes during the work** State: **5 of 5**

Keep unfinished behavior unavailable while integrating. A **feature flag** separates integration from making a feature available.



A feature flag can keep unfinished behavior unavailable while changes are integrated. The flag states and the combined behavior still need useful checks. Choose the strategy that fits the team and the change.

Principle · Choose a collaboration strategy that exposes incompatible assumptions early enough to act on them.

In short · Meet changed assumptions while the work is small.

When will the incompatible assumption become visible?

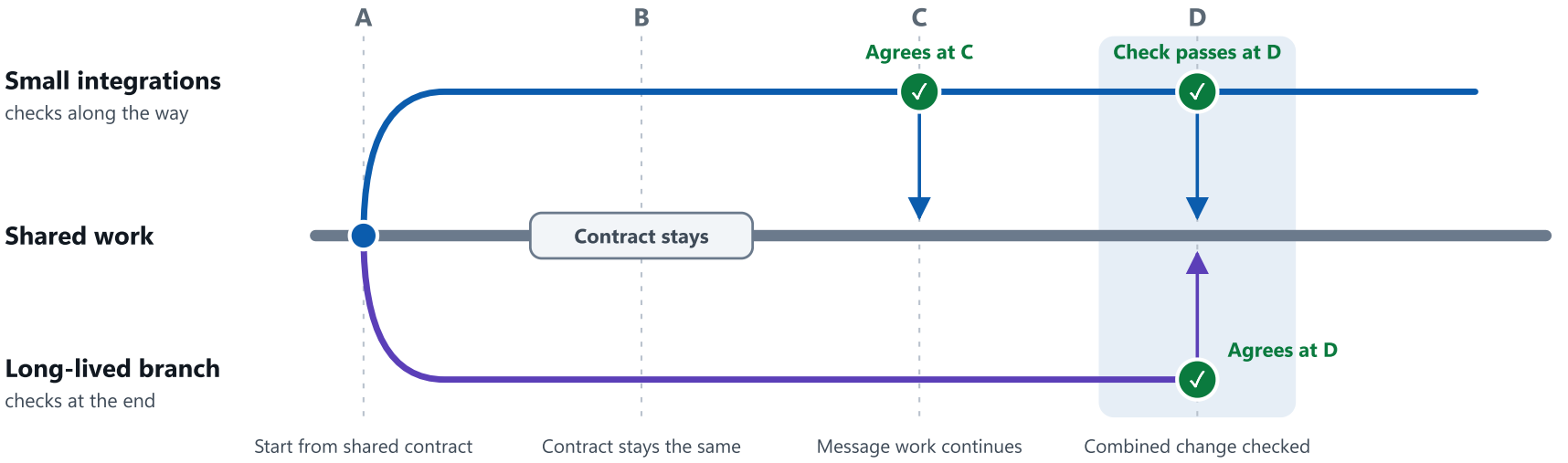
Panel 3 of 3 · Changed: the contract stays the same

Condition: **The shared message contract stays the same** State: **4 of 5**

At **D**, the later integration also agrees. Integration frequency affects when you discover a mismatch, not whether a mismatch exists.

THE CONTRACT BOTH LINES OF WORK DEPEND ON

A message stores its own duration limit.



SMALL INTEGRATIONS

The message assumption still matches the shared contract at this check.

LONG-LIVED BRANCH

The message assumption still matches the shared contract at this check.

With no changed assumption, both timelines agree at every check. Integration frequency decides when a mismatch becomes visible, not whether one exists.

The source is the same. Why did the answer change?

Panel 1 of 3 · Same source; the resolved dependency differs

Condition: **Only the resolved dependency changes** State: **4 of 6**

The same **120-minute** request now gets a different answer on the second machine.

DECLARED RUN

Source revision	`demo-r1`
Resolved dependency	`teaching-range 1.0`
Runtime	`Teaching runtime 1`
Configuration	`MAX_MINUTES = 120`
Data fixture	`duration-boundary-v1`
Build artifact	`rooms-demo-r1`

Allowed constraint: `teaching-range >=1.0,<2.0`

DEPENDENCY'S RANGE DECISION

```
1 def accepts(minutes, maximum):
2     return 30 <= minutes <= maximum
```

✓ **120 minutes → Accepted**

SECOND MACHINE

Source revision	`demo-r1`
Resolved dependency	`teaching-range 1.1`
Runtime	`Teaching runtime 1`
Configuration	`MAX_MINUTES = 120`
Data fixture	`duration-boundary-v1`
Build artifact	`rooms-demo-r1`

Allowed constraint: `teaching-range >=1.0,<2.0`

DEPENDENCY'S RANGE DECISION

```
1 def accepts(minutes, maximum):
2     return 30 <= minutes < maximum
```

✗ **120 minutes → Rejected**

teaching-range is an invented dependency. Its two versions deliberately differ at the upper boundary. ROOMS_TOKEN → secret store reference; store the reference with configuration, and keep the secret value outside the repository.

> The second machine selects teaching-range 1.1, which uses an exclusive upper boundary. A 120-minute request changes from accepted to rejected while source, configuration, runtime and data stay fixed. The dependency is invented for this comparison.

The source is the same. Why did the answer change?

Panel 2 of 3 · Changed: the configured maximum differs

Condition: **Only the configured upper limit changes** State: **4 of 6**

The same **120-minute** request now gets a different answer on the second machine.

DECLARED RUN

Source revision	`demo-r1`
Resolved dependency	`teaching-range 1.0`
Runtime	`Teaching runtime 1`
Configuration	`MAX_MINUTES = 120`
Data fixture	`duration-boundary-v1`
Build artifact	`rooms-demo-r1`

Allowed constraint: `teaching-range >=1.0,<2.0`

SECOND MACHINE

Source revision	`demo-r1`
Resolved dependency	`teaching-range 1.0`
Runtime	`Teaching runtime 1`
Configuration	`MAX_MINUTES = 90`
Data fixture	`duration-boundary-v1`
Build artifact	`rooms-demo-r1`

Allowed constraint: `teaching-range >=1.0,<2.0`

DEPENDENCY'S RANGE DECISION

```
1 def accepts(minutes, maximum):
2     return 30 <= minutes <= maximum
```

DEPENDENCY'S RANGE DECISION

```
1 def accepts(minutes, maximum):
2     return 30 <= minutes <= maximum
```

✓ **120 minutes → Accepted**

✕ **120 minutes → Rejected**

teaching-range is an invented dependency. Its two versions deliberately differ at the upper boundary. ROOMS_TOKEN → secret store reference; store the reference with configuration, and keep the secret value outside the repository.

> Both machines use teaching-range 1.0. Only MAX_MINUTES changes from 120 to 90, so the same 120-minute request is rejected. Change one relevant input at a time to explain the result.

The source is the same. Why did the answer change?

Panel 3 of 3 · Restore the declared inputs and repeat the teaching result

Condition: **Only the resolved dependency changes** State: **6 of 6**

Record resolved inputs, not only allowed version ranges. Reproducibility and dependency trust are separate questions.

DECLARED RUN

Source revision	demo-r1
Resolved dependency	teaching-range 1.0
Runtime	Teaching runtime 1
Configuration	MAX_MINUTES = 120
Data fixture	duration-boundary-v1
Build artifact	rooms-demo-r1

Allowed constraint: `teaching-range >=1.0,<2.0`

SECOND MACHINE

Source revision	demo-r1
Resolved dependency	teaching-range 1.0
Runtime	Teaching runtime 1
Configuration	MAX_MINUTES = 120
Data fixture	duration-boundary-v1
Build artifact	rooms-demo-r1

Allowed constraint: `teaching-range >=1.0,<2.0`

DEPENDENCY'S RANGE DECISION

```
1 def accepts(minutes, maximum):
2     return 30 <= minutes <= maximum
```

DEPENDENCY'S RANGE DECISION

```
1 def accepts(minutes, maximum):
2     return 30 <= minutes <= maximum
```

✓ **120 minutes → Accepted**

✓ **120 minutes → Accepted**

Restore the resolved teaching-range 1.0 dependency. A lockfile records the selected versions; a constraint such as >=1.0,<2.0 allows more than one selection.

teaching-range is an invented dependency. Its two versions deliberately differ at the upper boundary. ROOMS_TOKEN → secret store reference; store the reference with configuration, and keep the secret value outside the repository.

The resolved dependency set, runtime, configuration, data fixture and build record travel with the source revision. package-lock.json and uv.lock are examples of lockfiles. Keep secret values in a secret store; a placeholder reference belongs in the configuration record.

Principle · Reproducibility requires control of the relevant inputs, not just possession of the source files.

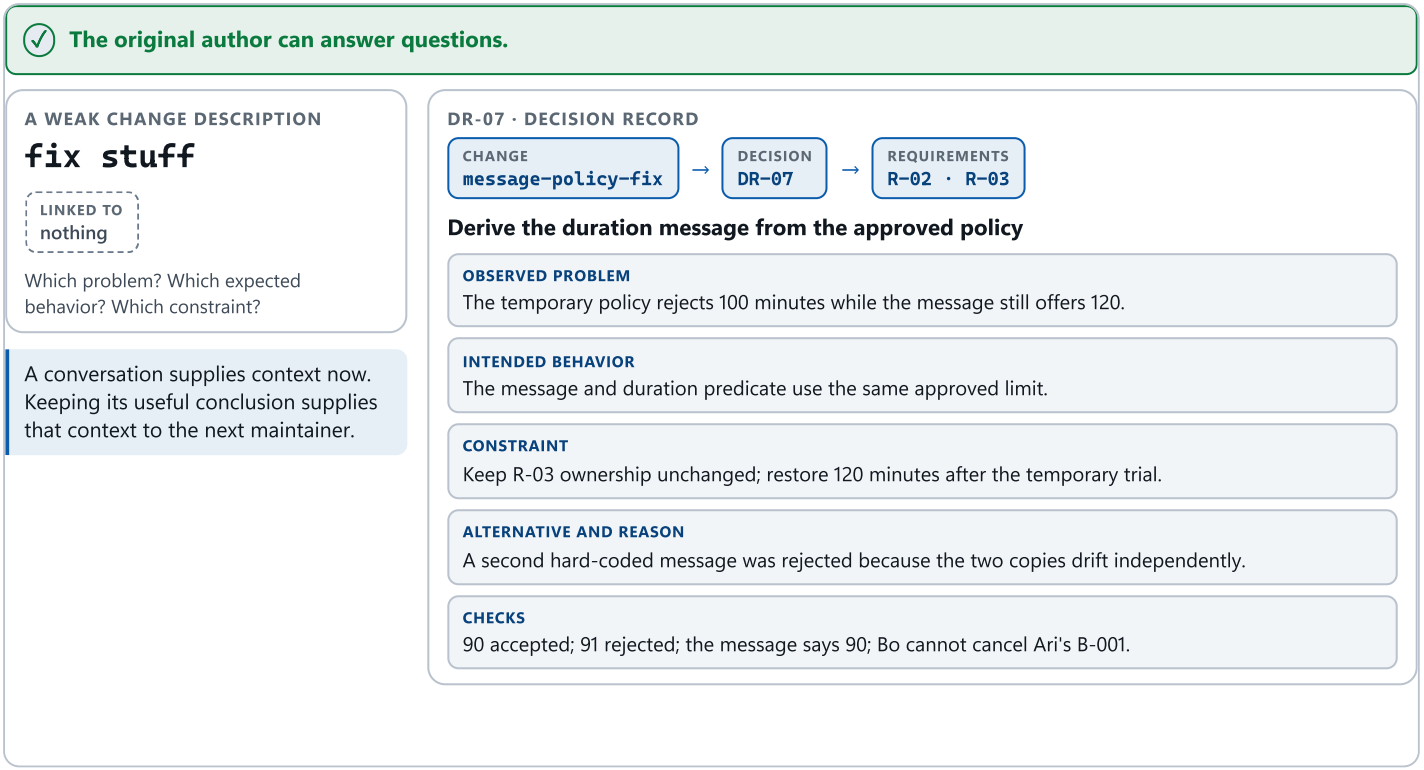
In short · Keep the inputs that explain the result.

Will “fix stuff” help you understand this change later?

Panel 1 of 2 · The useful record states the problem, behavior, constraint, and checks

Condition: **The original author can still answer questions** State: **6 of 6**

Link the relevant checks. A maintainer can now trace the purpose and reconsider whether its constraints still apply.



The title and DR-07 link the change to its reason. Preserve the conclusion and rejected alternative that help a later decision; a transcript of every conversation is unnecessary.

Principle · Documentation earns its place by preserving information that cannot be reliably reconstructed from the code.

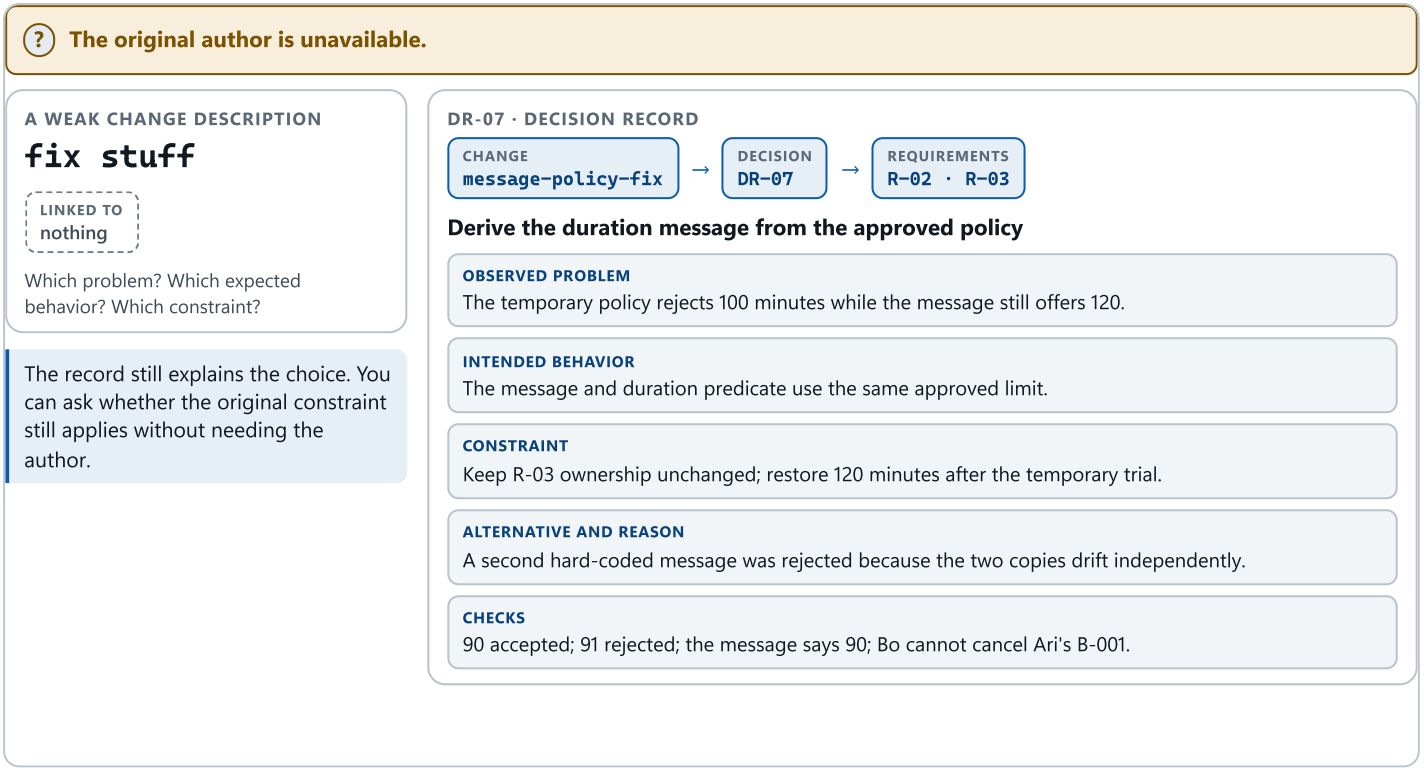
In short · Keep the reason that the code cannot tell.

Will “fix stuff” help you understand this change later?

Panel 2 of 2 · Changed: the author is no longer available

Condition: **The original author is unavailable.** State: **6 of 6**

Link the relevant checks. A maintainer can now trace the purpose and reconsider whether its constraints still apply.



The same durable record explains why the message follows Policy and why cancellation ownership remains unchanged. A future maintainer can reconsider the constraint rather than guessing from “fix stuff.”

Principle · Documentation earns its place by preserving information that cannot be reliably reconstructed from the code.

In short · Keep the reason that the code cannot tell.

Which change first broke the boundary?

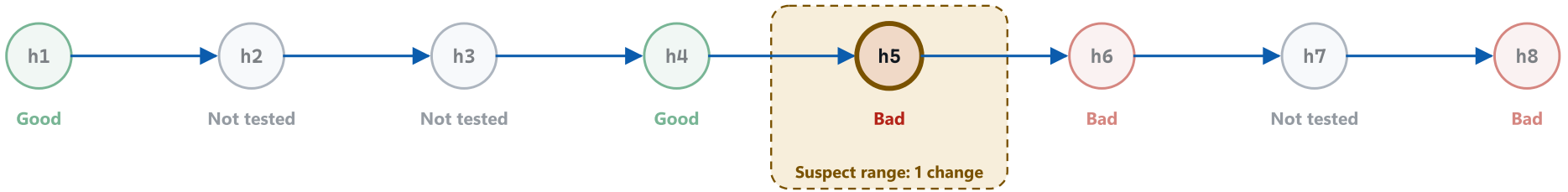
Test a midpoint, keep the part containing the failure, and repeat. This use of history is called bisection.

A separate linear diagnostic example; the h labels are not the c commit graph. Does the 91-minute boundary check fail under the temporary 90-minute rule? Good means 91 minutes is rejected; Bad means it is accepted. Known endpoints: h1 is Good; h8 is Bad.

Follow the repeated decisions

- h4: Good. Remaining candidates: h5, h6, h7, h8.
- h6: Bad. Remaining candidates: h5, h6.
- h5: Bad. Remaining candidates: h5.

After 3 checks, one change remains



> h5 removed the upper bound. The result identifies where to investigate; the rationale explains why.

The check is repeatable and the failure persists from its introduction through the bad endpoint.

Which missing evidence blocks this change?

Panel 1 of 3 · Passing checks leave a substantive authorization concern unanswered

Condition: **Checks pass; an authorization question remains** State: **2 of 6**

The open question is specific: can Bo cancel Ari's booking after this change?

Intended behavior	R-02 temporary 90-minute limit; restore the canonical limit afterward.	**Satisfied**
Boundary checks	90 accepted; 91 rejected.	**Satisfied**
Combined behavior	The displayed limit agrees with Policy.	**Satisfied**
Authorization concern	The cancellation question has no owner/non-owner result yet.	**Unanswered**
Human review	The R-03 authorization concern remains open until its evidence is reviewed.	**Concern open**
Integration check	The combined change passes its relevant checks.	**Satisfied**

? **Evidence missing**

> The boundary and message checks pass. The change still needs evidence for the open R-03 concern. The relevant cancellation check addresses that question.

Which missing evidence blocks this change?

Panel 2 of 3 · Changed: approval accompanies a failing integration check

Condition: **Review is approved; the integration check fails** State: **2 of 6**

Approval and a failing integration check describe different things. The failing behavior still needs repair.

Intended behavior	R-02 temporary 90-minute limit; restore the canonical limit afterward.	**Satisfied**
Boundary checks	90 accepted; 91 rejected.	**Satisfied**
Combined behavior	The displayed limit agrees with Policy.	**Satisfied**
Authorization concern	Ari can cancel B-001; Bo is rejected under R-03.	**Satisfied**
Human review	Required concerns resolved; the decision and rationale are recorded.	**Approved**
Integration check	The combined-change check fails. Approval cannot repair its behavior.	**Failing**

? **Evidence missing**

> Review approval describes the human judgment at that point. Repair the combined behavior and rerun the affected check before integrating.

Which missing evidence blocks this change?

Panel 3 of 3 · Resolve concerns, then restore the canonical policy

Condition: **Checks pass; an authorization question remains** State: **6 of 6**

Canonical baseline restored. 120 accepted; 121 rejected; the message agrees; ownership stays the same.

Intended behavior	R-02 restored: 30 to 120 minutes, inclusive.	**Satisfied**
Boundary checks	120 accepted; 121 rejected.	**Satisfied**
Combined behavior	Choose 30 to 120 minutes.	**Satisfied**
Authorization concern	Ari can cancel B-001; Bo is rejected under R-03.	**Satisfied**
Human review	Required concerns resolved; the decision and rationale are recorded.	**Satisfied**
Integration check	The combined change passes its relevant checks.	**Satisfied**

✓ **Canonical baseline restored**

LATER WEEKS USE THE CANONICAL BASELINE

R-02: 30 to 120 minutes, inclusive. 120 accepted; 121 rejected; message says 120. R-03 ownership is unchanged.

The temporary comparison is over. R-02 again accepts 30 to 120 minutes inclusive. The boundary checks accept 120 and reject 121; the message says 120. R-03 ownership remains unchanged. Week 8 examines how useful checks are chosen.

Principle · Integration readiness is a justified judgment about the change and its context, not a single green indicator.

In short · Resolve the concern behind each indicator.

Eight principles, six checks

Decide your answer, then open each check. The final question applies the same reasoning outside room booking.

Principles

1

History records relationships between changes; branch names are movable references, not the changes themselves.

In short: Follow the parents; watch the names move.

`week-07.html#/commit-graph`

2

A merge resolution must reconcile intent and verify the combined behavior, not merely remove conflict markers.

In short: Keep both intentions, then check the boundary.

`week-07.html#/conflict-and-intent`

3

Textual integration and semantic integration are different kinds of evidence.

In short: Check the promise across the combined files.

`week-07.html#/clean-merge-wrong-behavior`

4

Review is an evidence-based conversation about a change, not a vote on how convincing its author sounds.

In short: Make the purpose and the checks visible.

`week-07.html#/review-with-context`

5

Choose a collaboration strategy that exposes incompatible assumptions early enough to act on them.

In short: Meet changed assumptions while the work is small.

`week-07.html#/integration-frequency`

6

Reproducibility requires control of the relevant inputs, not just possession of the source files.

In short: Keep the inputs that explain the result.

`week-07.html#/same-code-different-result`

7

Documentation earns its place by preserving information that cannot be reliably reconstructed from the code.

In short: Keep the reason that the code cannot tell.

`week-07.html#/history-as-explanation`

8

Integration readiness is a justified judgment about the change and its context, not a single green indicator.

In short: Resolve the concern behind each indicator.

`week-07.html#/ready-to-integrate`

Checks: decide before reading the answer

1. What distinguishes a branch label from a commit?

> A branch label points to a commit and can move; the recorded commit represents a particular snapshot and ancestry.

2. Can two changes merge cleanly and still conflict?

> Yes. Their behavior or assumptions can contradict even when the edited text does not overlap.

3. What context should accompany a policy change?

> The intended requirement, affected behavior, relevant design constraints, and checks demonstrating the change.

4. Why record dependency inputs?

> The same source can run differently when its resolved dependencies or configuration differ.

5. What should resolve a substantive review disagreement?

> A clear claim, the evidence needed to decide it and an accountable decision; durable rationale should survive the discussion.

6. A library's code and documentation merge cleanly, but they describe different default behavior. Is the change ready?

> Resolve the intended contract and check the implementation and documentation against it. A textual merge detects some overlapping edits; it does not reconcile meaning across different files.

The change has a reason. How good are its checks?

Week 8 challenges the evidence that travels with a change.

Week 8 starts here

What makes a verification check useful, and what can a passing suite still fail to establish?

What carries forward

- The review packet: a clear claim, relevant checks, resolved concerns, and durable rationale.
- The canonical R-02 range and R-03 ownership rule, restored after the temporary comparison.
- A question about the quality of the checks, ready for Week 8.

In practice · examples from 2026

Git records history. GitHub and GitLab host pull or merge requests for review. package-lock.json and uv.lock record resolved dependencies. Ruff, ESLint, and Prettier handle routine linting or formatting before human review.

Sources

S01 IEEE Computer Society · SWEBOK https://www.computer.org/education/bodies-of-knowledge/software-engineering

S02 ACM / IEEE-CS / AAAI · CS2023 final report https://csed.acm.org/final-report/

S06 Pro Git · Basic Branching and Merging https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging

S07 Google Engineering Practices · Code Review https://google.github.io/eng-practices/review/reviewer/looking-for.html

S61 Git · Bisection https://git-scm.com/docs/git-bisect

S62 npm · package-lock.json https://docs.npmjs.com/cli/v11/configuring-npm/package-lock-json/

S63 Astral uv · The lockfile https://docs.astral.sh/uv/concepts/projects/layout/#the-lockfile

Terms used this week

Commit
A recorded snapshot with its parent relationships.

Branch reference
A name pointing to a commit; it can move as work advances.

Working tree
The saved files you are editing on your machine.

Staging area
The selected content prepared for the next Git commit.

Remote-tracking reference
Your local record of an observed remote branch position.

Diff
A view of removed and added content between two versions.

Merge commit
A commit with more than one parent, connecting histories.

Fast-forward
Moving a reference to a descendant already in the history.

Textual conflict
Edits the merge operation cannot reconcile automatically.

Semantic conflict
Combined behavior or assumptions that contradict each other.

Review packet
Purpose, diff, constraints, checks, and rationale for a change.

Version constraint
A rule describing which dependency versions are allowed.

Lockfile
A record of the selected dependency versions and resolution details.

Feature flag
A switch controlling whether a behavior is available.

Bisection
Narrowing a suspect history by testing a midpoint repeatedly.

Decision record
The decision, context, alternatives, and reasons worth preserving.